Milestone-2 Report: Safety Verification of Chiron IR using Constrained Horn Clauses

Project Overview

Program verification provides mathematical guarantees that software behaves correctly, unlike testing, which can find bugs but never prove their absence. While the Chiron framework offers support for dynamic analysis techniques such as fuzzing, symbolic execution, and spectrum-based fault localization, it currently lacks the capability to *prove* that a program satisfies a given specification for *all* possible inputs.

This project bridges the gap between testing and verification in Chiron by implementing a PDR-based safety verification engine. We target properties natural to turtle graphics programs, such as spatial bounds on the turtle's position and invariants over program variables.

Team Information

Roll No.	**Roll No.**	**Email ID**
Aditi Khandelia	220061	aditikh22@iitk.ac.in
Arush Upadhyaya	220213	arushu22@iitk.ac.in
Kushagra Srivastava	220573	skushagra22@iitk.ac.in

Source Code

The complete implementation is hosted at: `https://github.com/chiron-verification/milestone-2`

- **Milestone-1 commit:** `0f99ade6d4aa2516f9e47e013ad06d29e8b8a2f7`
- **Milestone-2 commit:** `<TBD>`

Core source files and responsibilities:

- `safety_properties.py`: Main verification API; parses user properties, applies checks/hints, and reports `PASSED/FAILED/UNKNOWN` with timing, and now refactored from an interactive CLI to callable `CHC_Verification(...)` with structured `ReturnValue`, timeout handling, property-scope control, and heading-grid pre-check/hint orchestration.
- `step_rules.py`: Encodes IR instructions into CHC transition rules used by the fixedpoint engine, with replaced trig bucket bounds with exact 15-degree trig encoding, introduced `BadHeading` rule generation, and added optimization-aware loop summarization and turn-safety shortcuts.
- `init_fixed_point.py`: Initializes solver state and base relations for default, specific, and universal modes, and now registers `BadHeading`, constrains universal initialization to heading-grid and integer user variables
- `z3_fixed_point.py`: Defines invariant/state-vector relations and mode-aware predicate structure and introduces the dedicated `BadHeading` relation in the same state signature as `Inv`.
- `variable_name_detection_in_IR.py`: Extracts variables from IR for building constraints and queries, and now introduces the shared `OptimizationLevel` enum and updates parser signatures/import path setup to align with the standalone `CHC-Verification` package layout.
- `heading_grid.py`: Encodes heading-grid checks to handle 15-degree alignment assumptions explicitly, and now added as a dedicated helper module exposing `heading_on_grid(h)` for reuse across initialization, transition encoding, and API-level assumptions.

- `optimization_helpers.py`: Provides static-analysis helpers for optimization decisions and loop simplification metadata, introduced to implement turn-safety dataflow, repeat-loop pattern detection, and summarizability checks used by `OptimizationLevel.BASIC`.

The correctness test suite (`correctness_tests/`) has substantially expanded across `test_default.py`, `test_specific.py`, and `test_universal.py`, along with many new `.tl` fixtures and shared helpers to improve coverage of arithmetic, control-flow, and heading-related properties.

In addition, a dedicated performance testing setup (`performance_tests/`) is added with mode-wise test runners, benchmark fixtures, and reusable harness/configuration files to measure solver behavior on deep nesting, wide-state, branch-heavy, and high-iteration workloads.

Current Progress

Overview of the progress made since the first milestone is as follows:

- **Pipeline refactoring into a callable API:** The verification flow is now exposed through `CHC_Verification(...)` with structured output fields (status, advice, timing, and per-property results) instead of an interactive-only CLI path.
- **Heading-grid safety checks integrated:** The verifier now models and queries heading-grid safety explicitly through a dedicated `BadHeading` relation before property checking.
- **Property scope feature added:** Properties can now be checked over all reachable states or restricted to terminating states only (`property_scope = "terminating"`).
- **Heading-grid hints integrated:** Users can either request verification of the heading-grid invariant (`check_heading_always_on_grid`) or force an assumption (`heading_on_grid_always`) for faster runs on hard cases.
- **Optimization pipeline added:** A `BASIC` optimization mode adds static IR analysis, turn-safety reasoning, and summarizable-loop compression before CHC querying, while `OptimizationLevel.NONE` retains the conservative baseline behavior.
- **Correctness tests:** An expanded test suite now covers a wider variety of programs and properties, including terminating-state properties, tests with heading-grid hints, and some tests with `OptimizationLevel.BASIC` to validate correctness under optimization.
- **Performance tests:** A performance test suite was designed to evaluate the impact of optimizations and hints on verification time and completion rates across a large set of programs.

See Figure 1 for a visual overview of the main verification pipeline stages, responsibilities, and data flow, and the subsequent sections for detailed implementation notes on each major feature addition and refactor. The details of correctness and performance testing can be found in the Tests section.

Implementation Details: Callable Verification API

File: CHC-Verification/safety_properties.py

The prior interactive `__main__` flow was replaced by a callable entry point:

```
CHC_Verification(
        file_name,
        mode,
        user_properties,
        params=None,
        property_scope="all",
        hints=["check_heading_always_on_grid"],
        timeout_ms=60_000,
        optimization_level=OptimizationLevel.NONE,
)
```

This API returns a `ReturnValue` object carrying: `status`, `heading_grid_safe`, `build_time`, `solve_times`, `passing_properties`, `failing_properties`, and `unknown_properties`. The implementation first validates mode, scope, hint format, and specific-mode parameters, then builds the fixedpoint, applies assumptions/prechecks, and evaluates properties in a controlled `eval` context with restricted builtins.

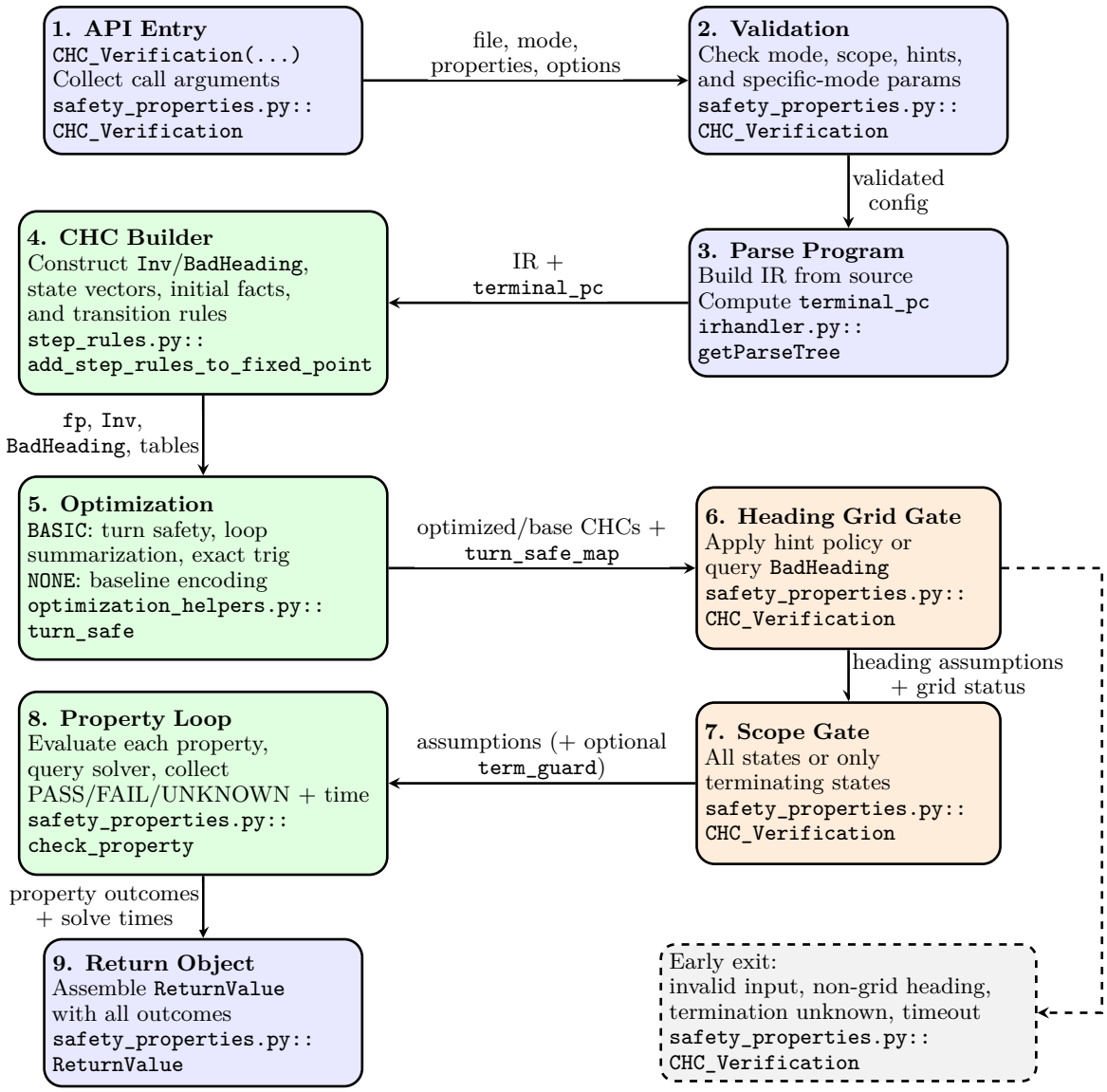

Figure 1: Main verification pipeline showing stage responsibilities and data artifacts flowing through parsing, CHC construction, heading/scope controls, and final result aggregation:

Implementation Details: Heading-grid Safety Check

Files:

1. *CHC-Verification/z3_fixed_point.py*
2. *CHC-Verification/init_fixed_point.py*
3. *CHC-Verification/step_rules.py*
4. *CHC-Verification/heading_grid.py*

The new pipeline introduces a dedicated relation `BadHeading(...)` at the invariant signature level and registers it in the fixedpoint context. Transition rules for turn commands can emit `BadHeading` consequences whenever `Not(heading_on_grid(next_heading))` is reachable, see Figure 2 for the workflow.

At API level, when checking is requested, the verifier queries `Exists(vars, BadHeading(state))`. The result semantics are: `sat -> heading_grid_safe = FAILED (overall UNKNOWN)`, `unsat -> heading_grid_safe = PASSED (continue)`, `unknown -> heading_grid_safe = UNKNOWN (overall UNKNOWN)`.

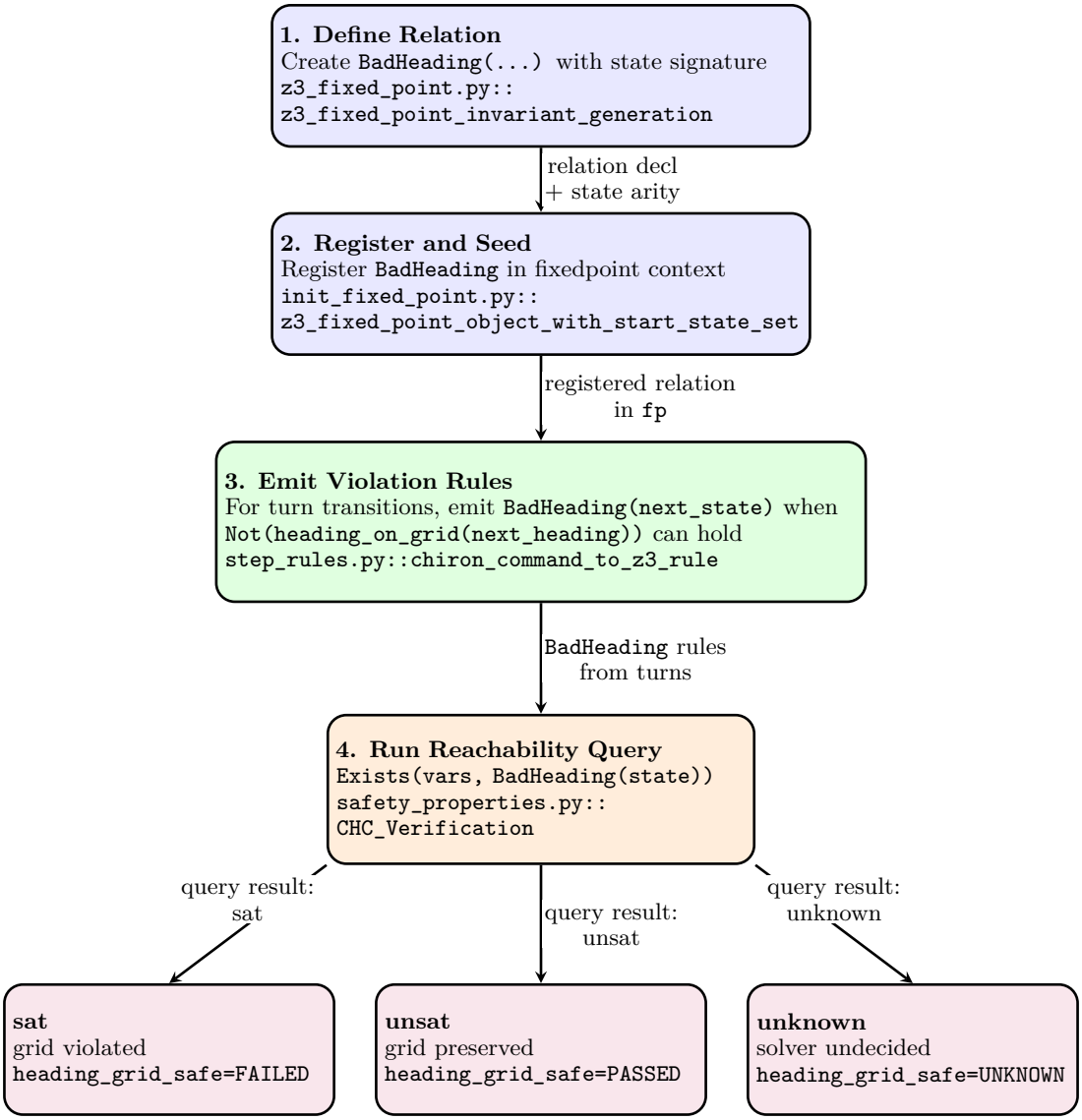

Figure 2: BadHeading workflow for proving whether heading remains on the 15-degree grid and mapping query outcomes to verification status:

This turns an implicit modeling assumption into an explicit proof obligation, improving interpretability of trig-sensitive verification outcomes.

Implementation Details: Property Scope

File: CHC-Verification/safety_properties.py

The new `property_scope` input supports `"all"` and `"terminating"`. For terminating-only checks, the implementation:

1. computes terminal PC as `len(ir)`;
2. checks reachability of terminating states using `Exists(vars, Inv(state) & term_guard)`;
3. returns `UNKNOWN` early if termination is unreachable/unknown;
4. conjoins `term_guard` into assumptions for all property queries.

This keeps a single property-checking loop while changing the semantic domain of verification in a principled way.

Implementation Details: Optimization Pipeline

Files: CHC-Verification/optimization_helpers.py, CHC-Verification/step_rules.py

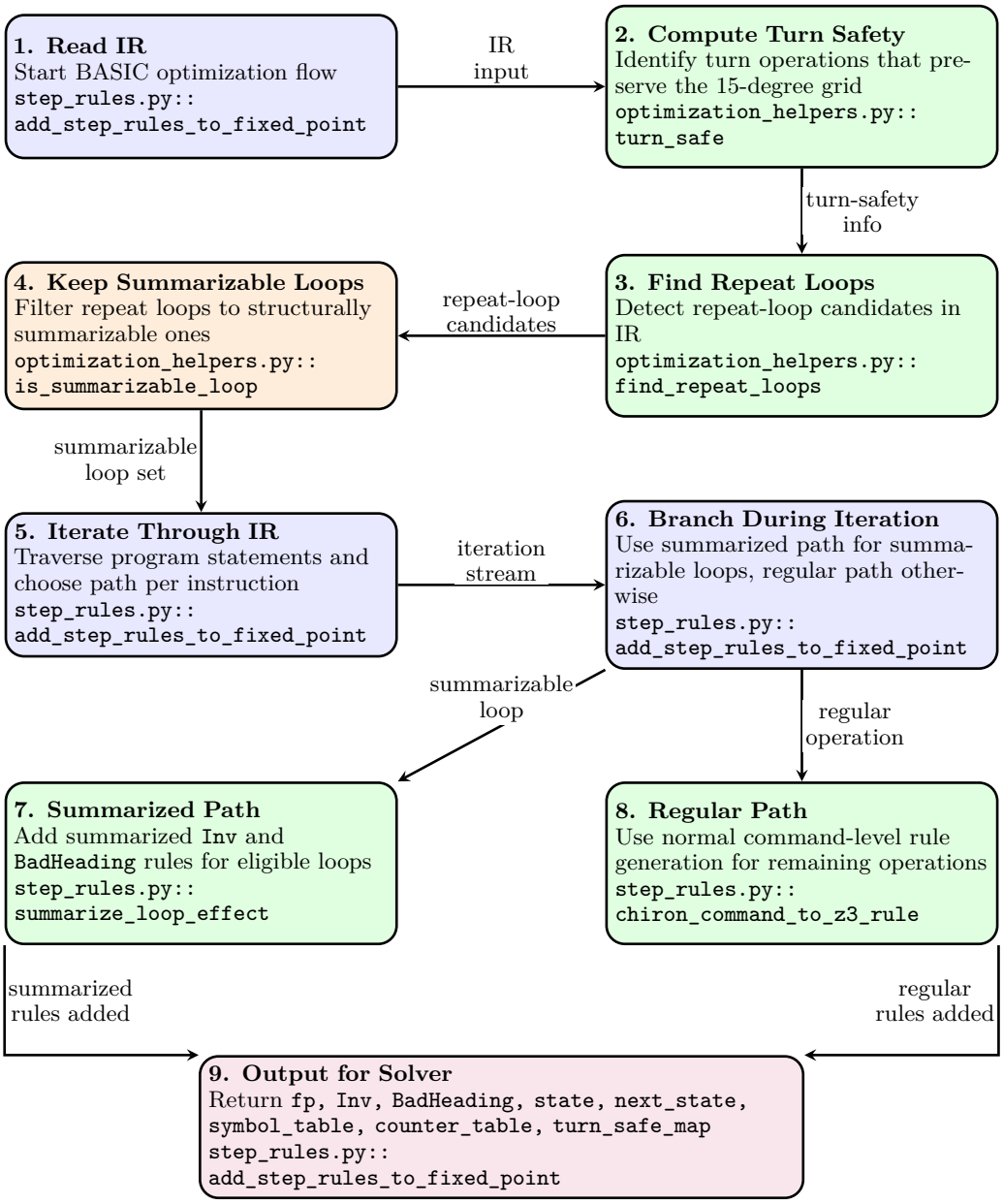

Figure 3: BASIC optimization workflow: read IR, compute turn safety, detect and filter repeat loops to summarizable loops, then during iteration emit summarized `Inv`/`BadHeading` rules for eligible loops and regular rules for other operations:

New optimazation layer with two supported modes. In `OptimizationLevel.BASIC`, the verifier computes a static environment over IR control-flow to detect turn operations that provably preserve the 15-degree grid (`turn_safe`). It also detects canonical repeat-loop shapes through `find_repeat_loops(...)` and admits only structurally safe loops via `is_summarizable_loop(...)`.

For those loops, `step_rules.py` synthesizes a summarized transition from loop-entry PC directly to loop-exit PC and emits any required summarized `BadHeading` obligations. For non-summarized code, the original

instruction-wise rule generation remains active. This preserves soundness while reducing rule count and symbolic clutter on common deterministic repeat patterns.

An additional semantic tightening is that movement trigonometric handling was upgraded from interval over-approximation buckets to exact 15-degree lookup values (`cos_sin_exact_z3`), aligning the transition encoding with the new heading-grid reasoning. The `OptimizationLevel.NONE` mode bypassed the optimization pipeline and performs standard verification. See Figure 3 for a visual overview of the optimization flow.

Implementation Details: Heading-grid Hints

File: CHC-Verification/safety_properties.py

Hints are parsed as a `StrEnum` with values `check_heading_always_on_grid` and `heading_on_grid_always`. The implementation validates hint type and value, then applies a precedence rule:

- if `heading_on_grid_always` is present, the solver skips the heading-grid query and directly injects `heading_on_grid(state[3])` into assumptions;
- otherwise, if `check_heading_always_on_grid` is present, the explicit `BadHeading` query is performed.

Thus, the user can choose between a sound check that provides explicit feedback on grid safety or a strong assumption that may speed up verification when grid safety is expected or irrelevant. The verifier can't guarantee soundness when skipping the check, so this is a user-controlled tradeoff.

API Usage Guide

The expected calling pattern is:

```
from safety_properties import CHC_Verification
from variable_name_detection_in_IR import OptimizationLevel

class UserProperty:
        def __init__(self, name, expr):
                self.name = name
                self.expr = expr

props = [UserProperty("x_nonneg", "x >= 0")]

result = CHC_Verification(
        "correctness_tests/programs/loop_accum.tl",
        "default",
        props,
        property_scope="all",
        hints=["check_heading_always_on_grid"],
        timeout_ms=60_000,
        optimization_level=OptimizationLevel.BASIC,
)

print("Result:", result.status)
print("Heading Grid Safe:", result.heading_grid_safe)
print("Build Time:", result.build_time)
print("Solve Times:", result.solve_times)
```

Operational notes:

- Use `specific` mode with a parameter dictionary string (e.g., `"'x': 10"`);
- Use `property_scope="terminating"` only when termination-focused properties are intended;
- Prefer `check_heading_always_on_grid` unless the program is known to be heading-grid safe or the check is not relevant, as provides a good balance of soundness and performance.

Tests

Correctness Tests

The correctness suite resides in `CHC-Verification/correctness_tests/` and is executed with `pytest` over `unittest.TestCase`-based test classes. The fixture set includes 64 `.tl` programs across the three modes, covering a wide range of semantic properties and edge cases. These fixtue programs can be found in `CHC-Verification/correctness_tests/programs/`.

Evaluation modules and responsibilities

File	Role in evaluation
`test_default.py`	Validates concrete-start-state verification in `default` mode, including arithmetic, geometric, pen-state, directional, heading-grid, trigonometric, and nested-loop behaviors.
`test_specific.py`	Validates parameterized initialization in `specific` mode, with boundary-sensitive outcomes under user-supplied parameter dictionaries.
`test_universal.py`	Validates symbolic/unconstrained initialization in `universal` mode, API-level validation errors, heading-grid handling, and terminating-scope properties.
`helpers.py`	Shared harness for loading programs, invoking `CHC_Verification`, building assertion contexts, and standardizing property checks.
`README.md`	Documents suite structure, mode coverage, fixture list, and execution commands.

Coverage snapshot

- Total collected correctness tests: **206**
- Mode split: **48** (default), **70** (specific), **90** (universal)
- Fixture programs in `programs/`: **64** `.tl` files
- Explicitly skipped tests: **6** timeout-marked heading/trig-heavy scenarios

Category coverage across modes

- **Default mode:** arithmetic invariants, geometric bounds, pen-state properties, directional/heading checks, and advanced nested control-flow fixtures.
- **Specific mode:** same semantic families under concrete parameter assignments, including multi-parameter and chained-update programs.
- **Universal mode:** unconstrained initial-state reasoning, relational arithmetic properties, API-validation behavior, and terminating-scope checks.

How to run

From repository root (recommended):

```
cd Chiron-Framework/ChironCore
source .venv/bin/activate
cd ../../CHC-Verification/correctness_tests
pytest -q
```

Useful variants:

```
# verbose mode
pytest -v

# collect-only sanity check
pytest --collect-only -q

# run one mode file
pytest test_default.py -q

# run one test class
```

```
pytest test_specific.py::TestSpecificArithmetic -q

# run one test case
pytest test_universal.py::TestUniversalAPI::test_invalid_mode_error -q
```

Performance Tests

The CHC verification pipeline is benchmarked to stress-test solver scalability under various optimization levels. The test suite resides in *Folder: CHC-Verification/performance_tests* and is executed with `pytest` using `pytest-xdist` for parallel execution. The fixture contains 21 `.tl` programs across the three modes and cover cases that can blow up the number of solver constraints. The fixture programmes can be found in *Folder: CHC-Verification/performance_tests/programs/*.

Test Infrastructure

The performance test suite is built on Python's `unittest` framework and reuses the shared helper infrastructure from Milestone 1. Each test case runs the verification pipeline across four configurations: two optimization levels (`NONE` and `BASIC`) combined with two hint modes (no hint, with hint).

File	Purpose
`tests/test_default.py`	Performance benchmarks in `default` mode, testing loop scaling, nesting depth, wide state, branching density, trigonometric motion, and property complexity.
`tests/test_universal.py`	Performance benchmarks in `universal` mode with unconstrained initial states.
`tests/test_specific.py`	Performance benchmarks in `specific` mode using explicit parameter assignments.
`tests/helpers.py`	Extended test infrastructure supporting timing measurements across all six configurations and CSV output generation.
`perf_results_*.csv`	Raw timing data output files containing build times, solve times, and verdicts for each test case.

Benchmark Programs

The performance suite includes 21 benchmark programs organized into six categories:

Program file	Purpose in evaluation
Loop Scaling (testing solver performance as loop iterations increase)	
`perf_repeat_10.tl`	Simple accumulator loop with 10 iterations; baseline for loop scaling.
`perf_repeat_50.tl`	Same accumulator with 50 iterations; tests medium-scale loop invariants.
`perf_repeat_100.tl`	Same accumulator with 100 iterations; tests large-scale loop invariants.
Nesting Depth (testing nested loop invariant synthesis)	
`perf_deep_nest_3.tl`	Three-level nested loops ($4^3 = 64$ total iterations); produces chained counters a, b, c.
`perf_deep_nest_4.tl`	Four-level nested loops ($3^4 = 81$ total iterations); adds counter d for deeper invariants.
Wide State (testing high-dimensional state tracking)	
`perf_wide_vars.tl`	Eight chained variables updated per iteration; tests 8-dimensional arithmetic invariants.
Branching Density (testing branching CFG complexity)	
`perf_many_branches.tl`	Six-iteration loop with four sequential if/else blocks per iteration; dense branching.

Trigonometric Motion (testing heading-dependent position updates)	
`perf_trig_10.tl`	`forward 10; right 90` repeated 10 times; tests trigonometric constraint scaling.
`perf_trig_20.tl`	Same motion pattern repeated 20 times; harder trigonometric reasoning.
Heading/Turns (testing heading normalization cost)	
`perf_turns_20.tl`	`right 345` repeated 20 times; heading visits only $\{0, 345\}$.
`perf_turns_50.tl`	Same turn pattern repeated 50 times; tests grid-check scaling.

Aggregate Results by Mode and Configuration

Tables 4, 5, and 6 present aggregate results for each verification mode across all six configurations.

Table 4: Aggregate results: **Default** mode (39 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	15	0	24	38.5%	0.141	2.74
BASIC	23	0	16	**59.0%**	0.100	2.16
NONE+H	23	0	16	**59.0%**	0.122	2.23
BASIC+H	23	0	16	**59.0%**	0.094	2.21

Table 5: Aggregate results: **Universal** mode (38 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	26	0	12	68.4%	0.206	4.51
BASIC	34	0	4	89.5%	0.863	4.15
NONE+H	30	0	8	78.9%	0.172	1.76
BASIC+H	36	0	2	**94.7%**	0.715	4.49

Table 6: Aggregate results: **Specific** mode (52 test cases).

Config	Passed	Failed	Skipped	Completion	Avg Build (s)	Avg Solve (s)
NONE	13	0	39	25.0%	0.312	1.09
BASIC	18	0	34	34.6%	0.262	1.19
NONE+H	20	0	32	**38.5%**	0.281	1.74
BASIC+H	19	0	33	36.5%	0.246	2.09

Figure 4 provides a visual comparison of verdict distributions across all modes and configurations.

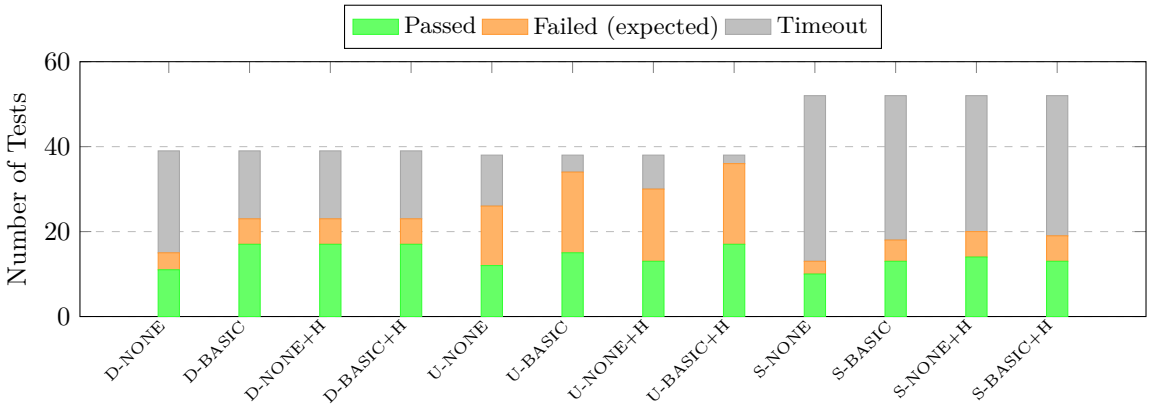

Figure 4: Verdict distribution across all modes (D=Default, U=Universal, S=Specific) and configurations. Green indicates proven properties, orange indicates expected counterexamples found, gray indicates solver timeout.

Timing Results

Tables present in *Files:* *CHC-Verification/performance_tests/perf_results_{default, specific, universal}.csv* present detailed timing results for each mode in all 4 configurations.

Build Time Analysis Across Verification Modes

Figure 5 shows average build times across the three verification modes for each configuration.

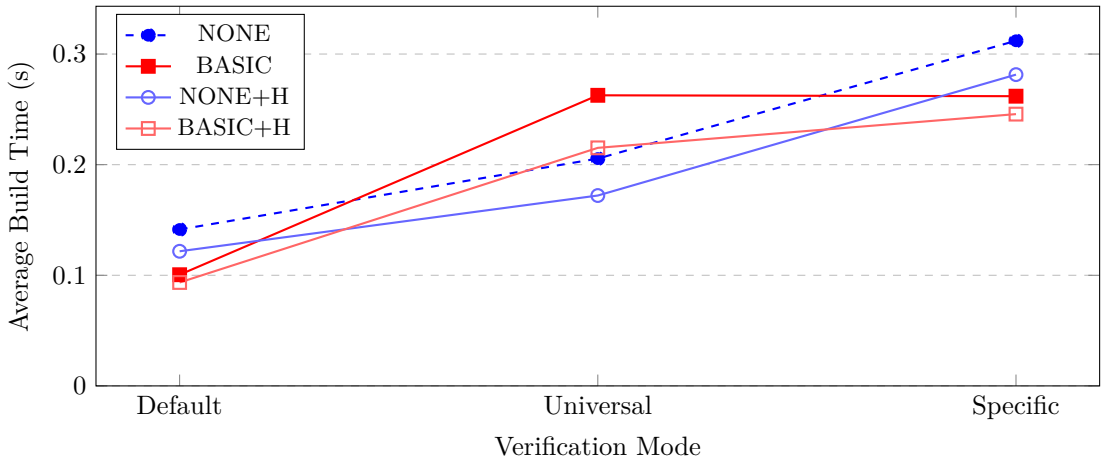

Figure 5: Average build times across verification modes. Default mode shows fastest build times; Specific mode has highest due to parameter processing overhead.

Solve Time Analysis Across Verification Modes

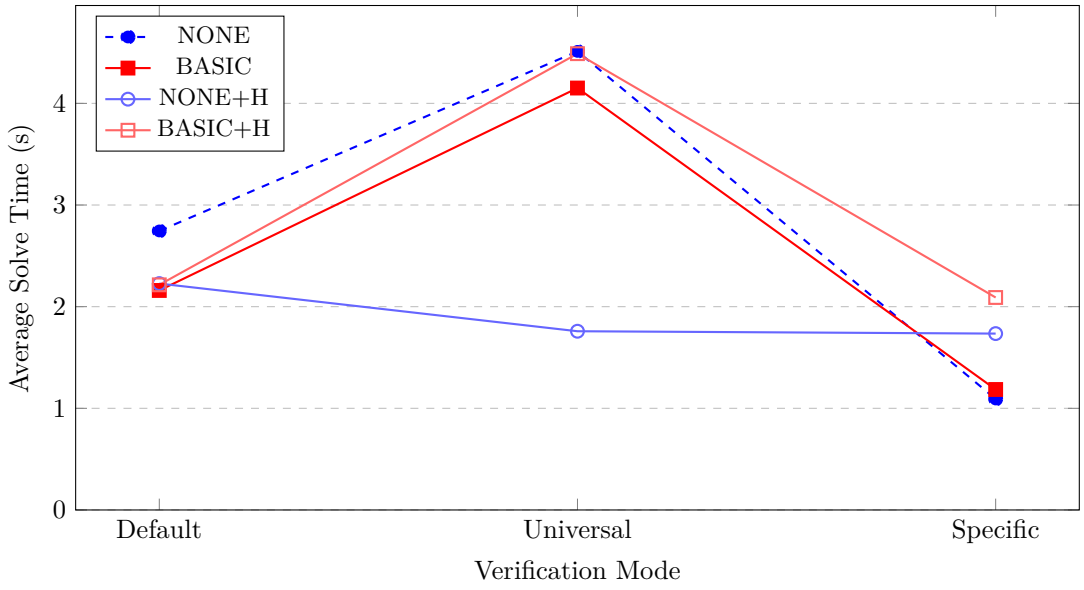

Figure 6: Average solve times across verification modes. Universal mode exhibits highest solve times due to unconstrained initial states.

Optimization Benefits

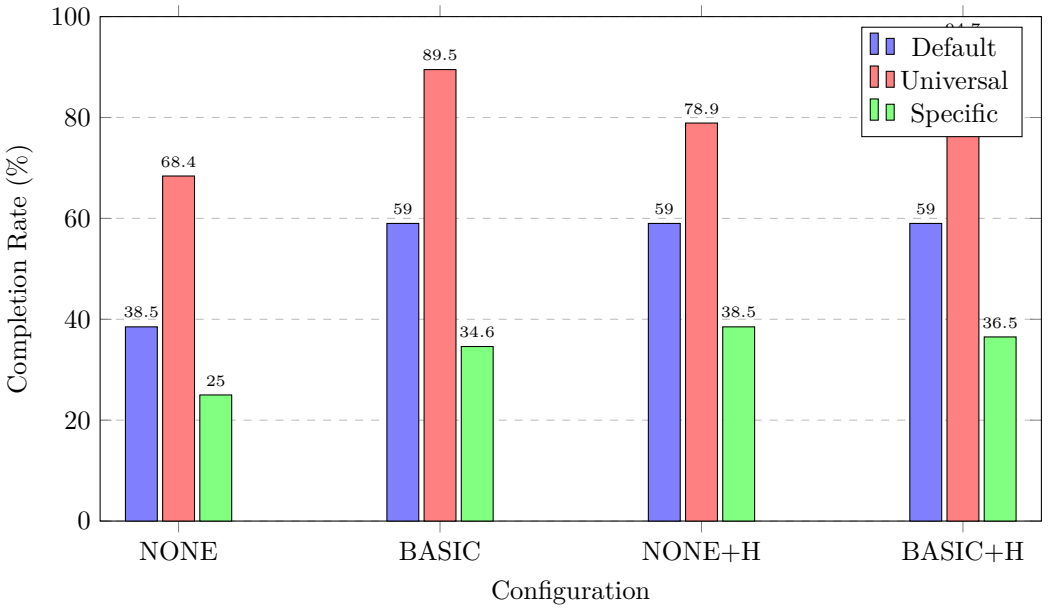

Figure 7: Test completion rates by configuration. `BASIC+H` achieves 94.7% completion in Universal mode—the highest across all mode-configuration combinations.

- **`BASIC` provides consistent improvement.** In Default mode, completion jumps from 38.5% (NONE) to 59.0% (BASIC); in Universal mode, from 68.4% to 89.5%.
- **Mode-dependent behavior.** Universal mode shows the largest improvement with `BASIC` (68.4% → 89.5%), while Specific mode shows minimal benefit (25.0% → 34.6%).

Timeout Reduction on using Hints

Table 7: Timeout reduction rates with heading hints enabled.

Mode	NONE → NONE+H	BASIC → BASIC+H
Default	**33.3%** (24→16)	0.0% (16→16)
Universal	33.3% (12→8)	**50.0%** (4→2)
Specific	17.9% (39→32)	2.9% (34→33)

Tests Uniquely Solved by Hints

Table 8: Number of tests uniquely solved when hints are enabled (timeout→success).

Mode	NONE	BASIC	AGGRESSIVE
Default	8	1	8
Universal	5	2	7
Specific	7	2	6
Total	**20**	**5**	**21**

- **Combining `BASIC` with `HINTS` is optimal.** It achieves the highest completion rates (59.0% Default, 94.7% Universal, 36.5% Specific).
- **Enable hints for trigonometric programs.** When verifying programs with heading-dependent motion (`forward`/`backward`), hints provide a significant timeout reduction.

Overall Performance

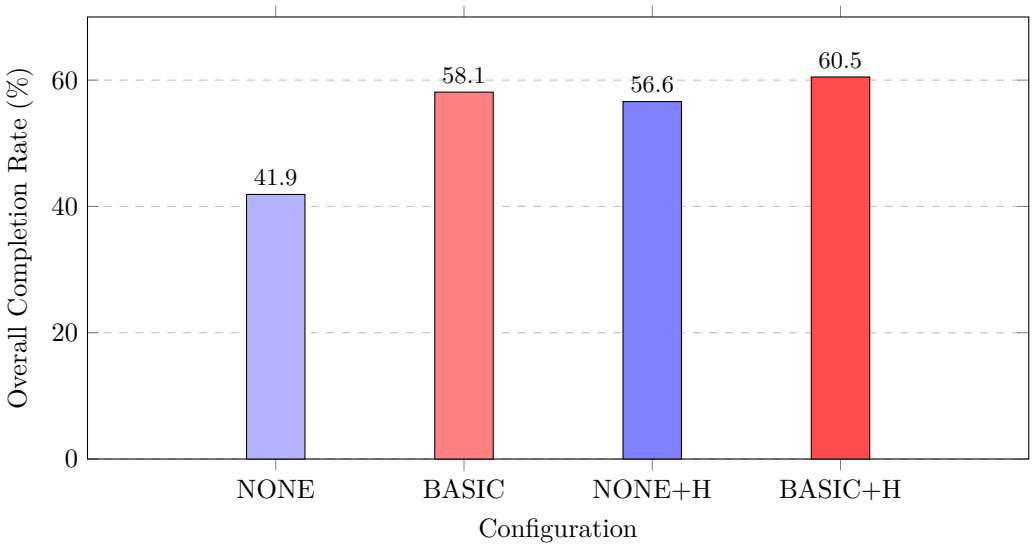

Figure 8: Overall completion rates across all 129 tests. `BASIC+H` achieves the highest overall rate (60.5%).

Running the Performance Tests

```
cd Chiron-Framework/ChironCore/CHC_Verification/performance_tests
pytest test_default.py -v      # Default mode benchmarks
pytest test_universal.py -v    # Universal mode benchmarks
pytest test_specific.py -v     # Specific mode benchmarks
```

Results are written to `perf_results_<mode>.csv` for analysis.

Analysis

Strengths of the implementation

- **Broad verification coverage:** The pipeline supports default, universal and specific modes, now augmented with terminating-state properties (e.g., the turtle eventually reaches a target), enabling richer correctness questions.
- **Improved solver efficiency:** The new `OptimizationLevel.BASIC` path for universal properties reduces solver overhead and makes universal queries feasible on more complex programs, specifically those with loops, and the new heading-grid safety hints significantly reduce timeouts for geometric programs.
- **Heading-grid safety hints:** The introduction of heading-grid-based safety hints significantly reduces solver timeouts for geometric programs where the user is sure the turtle will only turn in multiples of 15 degrees.
- **Stronger empirical validation:** Performance tests and visual analyses help compare the impact of optimizations as heading-grid safety hints.
- **Improved test coverage:** The test suite now includes more diverse cases, covering both geometric and arithmetic properties as well as terminating state properties, and has been expanded to include stress tests for deeper loops and complex properties.

Limitations of the implementation

- **Limitation to heading-grid safety:** Movement commands that go out of the heading grid (e.g., 22.5 degree turns) lead to unknown verdict irrespective of the property.
- **Nonlinear arithmetic fragility:** Multiplicative constraints and non-linear relationships can still produce `UNKNOWN` results or long runtimes, especially when combined with geometric constraints.

- **Structured Geometric Reasoning:** Despite the hints and optimizations, some programs and properties involving complex geometric reasoning (e.g., intricate heading interactions) still lead to timeouts, indicating room for further encoding-level improvements.
- **Conservative Approach to Optimization:** The optimizations are designed to be sound but may miss opportunities, like in heading-grid safety check, a multiplication will only be considered a multiple of 15 if both the terms are multiples of 15, which is a sound but conservative check that may miss some cases where the product is a multiple of 15. Similarly for loop summarization, the current implementation only summarizes loops with a single update pattern, which may miss opportunities for summarization in more complex loops.
- **Limited scalability validation:** While the test suite has grown, scalability for programs with extremely large loop counts or deeply nested structures remains unproven.

Future Plans

Milestone summary

Milestone 1 Literature review and naive CHC pipeline - *completed*: CHC theory studied, Chiron-to-CHC semantics formalised, Z3 Fixedpoint playground implemented. First end-to-end pipeline with basic CHC encoding and property checking developed, supporting three modes: `default`, `universal`, `specific`, but with limited optimizations and test coverage.

Milestone 2 End-to-end robust CHC verification pipeline - *completed*: This milestone focused on strenghtening the full IR-to-CHC translation pipeline, adding support for checking property at program termination, restricting to 15-degree mutliple grid for determinsm in restricted case and support optimization for special cases.

Final Submission Further optimization and user-friendly API - *pending*: The final milestone will focus on improving the pipeline's efficiency through further encoding optimizations and solver parameter tuning. Additionally, efforts will be directed towards developing a user-friendly API to enhance accessibility and usability for end-users. This phase will also include final documentation, a polished test suite, and a reproducible end-to-end tool.

Planned work

(a) Optimizations

We plan to work on additional optimizations to further enhance the performance of the CHC encoding and solver interaction, and user hints to guide the solver. These include :

- **Property-aware State Projection:** For many properties, only a subset of the program's state space is relevant. We will explore techniques to project the state space onto relevant dimensions, reducing the complexity of the CHC encoding and improving solver performance.
- **Strengthening of Turn-Safety Check:** Right now, the turn-safety check is exceedingly conservaive, which leads to a large number of unknown results. We will investigate ways to strengthen this check, potentially by incorporating additional program invariants or leveraging more precise abstractions of the program's behavior.
- `focus_vars = [ ]`: Allowing users to specify a subset of variables that are most relevant to the property being checked, enabling the solver to focus on a smaller state space.
- **Recursive summarization for perfectly nested affine loops:** If the inner loop is summarizable, we can summarize it first into one affine state transformer, and then summarize the outer loop with the inner loop summary as a single state transformer. This can significantly reduce the size of the CHC encoding for nested loops.

(b) User-friendly API development

To make the pipeline more accessible, planned work includes:

- **Variable Domain Declarations:** Instead of constraining the user to declare a single initial value for variables in the specific mode, we will allow users to specify a domain of possible initial values. This will enable more comprehensive verification while still maintaining a manageable state space.

- **Semantic Property Specification:** Right now, the users need to specify the properties in the form of raw property strings, which can be difficult to use for non-experts. We aim to add helpers like `heading_in(range)`, $position\_in\_box(xmax, xmin, max, ymin)$, $pen\_is\_down()$ etc. to allow users to specify properties in a more intuitive and domain-specific manner, abstracting away the underlying CHC encoding details.

(c) Final integration and deliverables

The final phase will consolidate all components into a cohesive tool, including:

- Stabilizing CLI outputs and ensuring reproducibility across environments.
- Completing the final report and preparing the project poster.
- Delivering a fully documented and user-friendly verification framework.

AI Use in the Project

In this project, we utilized AI tools to enhance our workflow and improve the quality of our work.

1. **Are you using AI for coding your implementation?** Yes, we used AI tools in this project.
2. **If yes, what models are you using?** We used models like OpenAI's GPT-5.2 Codex and GitHub Copilot.
3. **If using AI, how are you using AI and what percentage (approx) of your code is AI generated?** We used AI primarily for generating boilerplate code, suggesting improvements, and debugging. Approximately $40\% - 50\%$ of our code was generated or assisted by AI tools.
   - AI was primarily used for generating boilerplate code and usage of correct syntax, which was then reviewed and modified by us to fit our specific requirements.
   - We also used AI for testing, where it helped in generating test cases and suggesting improvements to our existing tests.
   - Additionally, AI was used for the generation of documentation and figures in the report, where it assisted in summarizing the directory structure and usage guide, as well as making the figures in the report with careful and structured instructions about the layout.

Declaration

- We declare that the work presented in this report is our own and has been carried out in accordance with the guidelines set out by the course.
- We have not copied any part of this report from any other source, and we have not submitted this report to any other course or institution for assessment.
- We understand that any violation of this declaration may result in disciplinary action.